Ders 2 Genişletilmiş Notu: Metin Görüntüleyici Örneğiyle Analiz Modeli ve Temel Tasarım Kavramları

Ders 2 Genişletilmiş Notu: Metin Görüntüleyici Örneğiyle Analiz Modeli ve Temel Tasarım Kavramları

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Tasarım öğretilmez, öğrenilir
- 2. Örnek problem: metin görüntüleyici
- 3. Bileşenleri bulmak: GUI elemanları ve DosyaYöneticisi
- 4. Gösterici ve kaydırma çubuğu gereksinimleri
- 5. Kullanım senaryoları ve davranış modellemesi
- 6. Sınıf diyagramı: özellikler, işlemler, parametreler
- 7. Görünen özellikler (percept) ve paydaşlar
- 8. Bileşenler arası ilişkiler
- 9. Uç durumlar ve analiz modelinin özeti
- 10. Genel tasarım kavramları: tasarım, mühendislik, zanaat, sanat
- 11. Yazılım tasarımı ve programlama
- 12. Mimari tasarım ve detaylı tasarım
- 13. Yazılım tasarımı yaklaşımları ve ortak üç yön
- 14. Tasarım ikilemleri ve geçerleme
- 15. Belgelendirme
- 16. Tasarımda anahtar kavramlar ve tasarım felsefesi
- Örnek Sorular

Ders iki parçadan oluşur. İlk parçada çok basit bir metin görüntüleyici üzerinden bir problemin analiz modelinin nasıl kurulduğu adım adım gösterilir. İkinci parçada tasarım, mühendislik ve yazılım tasarımıyla ilgili genel kavramlar ele alınır. Kullanılan slayt destesi 43 slayttır (ekrandan, 00:20:00).

1. Tasarım öğretilmez, öğrenilir

Hoca dersin asıl amacını yeniden hatırlatır: yazılım mimarilerini, kullanım şekillerini ve uygulamalarını öğrenmek. Ancak bir yazılımı yüksek seviyeli bir mimariyle kurgulayabilmek için o yazılımın kendisinin de düzgün tasarlanmış olması gerekir. Bu yüzden mimari kadar tasarım da doğru yapılmalıdır.

Hocaya göre tasarımı öğretmenin belirli bir yolu yoktur. “Önce şunu şununla topla, sonra bununla çarp” gibi net bir tarif verilemez. Hoca ancak tavsiye verebilir. Tasarım, öğretilmekten çok öğrenilmesi gereken bir süreçtir. Bu yüzden ders, hazır bir yöntem yerine küçük bir örnek üzerinde düşünme biçimini göstererek ilerler.

2. Örnek problem: metin görüntüleyici

Örnek problem, bir metin editörü bile değil, bir metin görüntüleyicidir (text view). Herhangi bir metin dosyasını diskten okuyup ekranda satır satır gösteren basit bir araçtır. Dosya yalnız

metinden oluşur.

Problemin önemli bir kısıtı vardır. Normalde .NET Framework'te ya da Java Swing'de hazır bir dosya görüntüleme bileşeni koyup işi bitirmek mümkündür. Bu örnekte böyle bir bileşen yoktur. Grafik arayüz araçları dosyayı açıp içeriğini gösteren bir bileşen sunmaz. Çözüm temiz bir şekilde yapılandırılarak bileşenlerden kurulacaktır.

3. Bileşenleri bulmak: GUI elemanları ve DosyaYöneticisi

3.1 İlk soru: hangi GUI elemanları gerekir?

Slayttaki ilk soru şudur (ekrandan, 00:17:15): Swing ya da SWT gibi bir GUI kütüphaneniz olsun. MetinGörüntüleyici tasarlamak için hangi atomik GUI elemanlarına ihtiyaç duyarsınız? Seçenekler Checkbox, Window, Progress Bar, Combo Box, Scroll Bar ve Radio Button'dır.

Sınıftan cevap gelmeyince hoca kendisi seçer:

- **Checkbox:** Tikleme işaretine büyük olasılıkla ihtiyaç yoktur.
- **Window:** Gerekir, çünkü metin ekranda bir pencere içinde gösterilecektir.
- **Progress Bar** ve **Combo Box:** Gerekmez.
- **Scroll Bar:** Gerekir. Metin dosyasının ne kadar uzun olduğu bilinmez ve ekranda gösterilebilecek satırın bir sınırı vardır (örneğin 24 satır). Sonrası için ekranı kaydırmak gerekir.
- **Radio Button:** Gerekmez.

Sonuç: bir pencere ve bir kaydırma çubuğu yeterlidir (ekrandan, 00:20:00).

3.2 Metni kim sağlayacak?

Görsel bileşenler hazırdır, ama metne de erişmek gerekir. Dosya sisteminde verilen adreste duran dosyayı açıp içeriğine erişecek bir bileşen GUI'de yoktur. Slayttaki ifadeyle: "Peki metni bize kim sağlayacak? GUI'de yok. Ama metne erişebilmeliyiz." (ekrandan, 00:20:00). Bu yüzden üçüncü bir bileşen eklenir: **DosyaYöneticisi** (File Manager).

Hoca bu bileşenleri yalnız düşünerek bulduğunu vurgular. Sorulan soru şudur: elimde ne var, neleri kullanabilirim, hangi şeyler bana hazır olarak verilmemiş?

3.3 DosyaYöneticisi kısıtları

Slayttaki kısıtlar şunlardır (ekrandan, 00:20:00):

- Dosyanın tüm içeriği hafızada tutulamaz.
- Kalanı için diske erişip okuma yapmak gerekir.
- İşletim sistemi seviyesinde dosyaya satır satır erişim vardır. Kütüphanedeki satır okuma modülü, istendiğinde belirli bir uzunluğa kadar ardışık satırları çekip verir. Dosya açma ve kapatma gibi ayrıntıları da halleder.

Hoca birinci kısıtı şöyle açar: dosyayı açıp okuyup kapatarak tamamını belleğe almak mümkün değildir. Dosya 4 GB ya da daha büyük olabilir. Uygulamanın 2 GB belleği olan bir sanal makinede çalışması beklenebilir. Hoca normalde dosyayı bir değişkende tutup satır satır işlemeyi tercih edeceğini, ama bunun burada mümkün olmadığını söyler. Örneğin kullanıcı kaydırdığında sıradaki 80 satırı diskten okuyacak bir bileşen gerekir. Satır okuma modülü "bana 80 satır ver", "bana 50 satır ver" biçiminde bir parametreyle kullanılabilir.

Hoca burada yapılan işi de adlandırır. Dosya sistemine erişmek uygulamanın bir sorumluluğudur. Bu sorumluluk uygulamanın içine gömülmemiş, ayrı bir bileşen olarak, hizmet ya da servis olarak alınmıştır. Böylece bileşenin girdi ve çıktı beklentileri de tasarlanmaya başlanmıştır. Hoca ayrıca alan (domain) ve iş analizinden gelen bu tür işlevsel ve işlevsel olmayan kısıtların mimari tasarımı da etkileyeceğini belirtir.

4. Gösterici ve kaydırma çubuğu gereksinimleri

4.1 Gösterici

Pencereye (viewer, viewport) **Gösterici** adı verilir. Slayttaki gereksinimler (ekrandan, 00:40:00):

- Tamsayı (integer) satır sayısı gösterebiliyor.
- 1 ile 100 satır arasında boyutu değiştirilebiliyor.

Bu iki gereksinim, pencere ile dosya yöneticisinin işlerini birbirine bağlar. Pencere 20 satır gösterecek şekilde boyutlandırıldıysa dosya yöneticisinden 20 satır beklenir. Kullanıcı pencereyi aşağıdan çekip 50 satır gösterecek hâle getirirse 30 satıra daha ihtiyaç vardır. Bu satırlar nasıl istenecek? Baştan 50 satır mı istenecek, yoksa "20 bende var, 30 daha ver" mi denecek? Hoca bunların hepsinin tasarımcının vermesi gereken kararlar olduğunu söyler.

Hoca örneği basit tutmak için bazı şeyleri kapsam dışı bırakır. Yazı tipi ve punto değiştirilemez; Word'de olduğu gibi farklı puntolar kullanılsaydı ekranda gösterilebilecek satır sayısı değişirdi. Örnek, terminalde çalışan vi gibi sabit yazı tipli bir görüntüleyici olarak düşünülür. Punto değişebilseydi pencere ve dosya yöneticisi kendi işlerini bütün olarak yapmaya devam ederdi, ama araya punto ve boyuta göre kaç satır çekileceğini hesaplayan bir dönüştürücü bileşen gerekirdi.

4.2 Kaydırma çubuğu

Slayttaki tanım şöyledir (ekrandan, 00:30:00):

- Dosyanın değişik yerlerini gösterebilmek için kaydırma çubuğuna ihtiyaç vardır.
- Alışılmış biçimde pencerenin sağında duran, yukarı aşağı hareket ettirilebilen parçası olan bir çubuktur.
- Hareket eden parçaya **kulp**, kulbun hareket ettiği alana **tabla** denir.
- Kullanıcı tablada kulbu hareket ettirerek dosyada gördüğü pozisyonu değiştirebilmelidir.
- Kulbun bulunduğu pozisyona göre Göstericide dosyanın ilgili parçası gösterilmelidir. Kulp en üstteyken dosyanın başı, en alttayken sonu gösterilmelidir.
- Kulbun boyutunun tabla boyutuna oranı, dosyanın Göstericide gösterilen parçasının boyutuna denk düşer. Dosyanın tüm içeriği Göstericiye sığıyorsa kulp tüm tablayı kaplar. Dosya çok büyükse ya da Gösterici çok küçükse incecik bir kulp görülür.

  Ders sonrası not: Hoca anlatırken (00:25:44) kaydırma çubuğunun "mesela solda" durduğunu söyler. Slayttaki tanım "pencerenin sağında duran" çubuktur. Bu büyük olasılıkla bir dil sürçmesidir.

Yatay kaydırma da kapsam dışıdır. Satır genişliğinin değişmesi istenseydi yatay bir kaydırma çubuğu ve başka bir bileşen gerekirdi. Hoca bu durumda pencerenin bu işi üstüne almaması, işin arada başka bir bileşen tarafından sağlanması gerektiğini söyler. İlke "mümkün olduğunca herkes kendi işini yapsın" ilkesidir. Hoca, çok basit bir problemin bile düzgün tasarlanmak istendiğinde ne kadar ayrıntılı düşünme gerektirdiğini vurgular.

4.3 Kaydırma çubuğunda karar gerektiren durumlar

Hoca kaydırma çubuğunda hemen akla gelmeyen durumları sıralar:

- Kulp en tepeye çekilirse ve ekran 20 satır gösteriyorsa dosyanın 1. ile 20. satırları gösterilir. Bu kolaydır.
- Kulp en alta çekilirse dosyanın sonu gösterilmelidir. Son satır en tepeye alınamaz; sondan önceki 20 satır alınmalıdır. Bu yüzden kulbun konumuna göre bir hesap yapmak gerekir.
- Daha zor durum: kulp en alttayken, yani dosyanın son 20 satırı gösterilirken, pencere 40 satır gösterecek şekilde büyütülür. Yazılım son 20 satırı gösterip altında 20 boş satır mı bırakmalı, yoksa son 40 satırı mı göstermeli?

Hoca bu son sorunun cevabını vermez. Kararı analiz ve tasarımda tasarımcının vereceğini söyler. Dersin ilerleyen kısmında (01:17:27) bu konuya döner ve iki davranışın da doğru olabileceğini, ama birinin varsayılıp davranışın ona göre modellenmesi gerektiğini belirtir.

Kulbun boyutu için de bir hesap gerekir. Bunun için dosyanın boyutunu ve pencerenin boyutunu bilmek ve oransal bir hesap yapmak gerekir. Kulp bütün tablayı mı kaplayacak, yoksa dosya büyüdükçe mi küçülecek?

5. Kullanım senaryoları ve davranış modellemesi

Bileşenler bulunduktan sonra davranışlarına bakılır. Hoca buna davranış modellemesi der. Mimari tasarım parçaları ortaya konmuştur, ama bunların nasıl çalışacağı da ortaya konmalıdır.

Senaryo, planlanan çözümün tamamen kullanıcı gözüyle nasıl kullanılacağını düşünüp ortaya koymaktır. Örneğin kaydırma çubuğu kaydırıldığında ya da gösterici yeniden boyutlandırıldığında sistemin adım adım ne yapması beklendiği senaryo olarak yazılır. Böylece davranışlar sonra ilgili bileşenlere atanabilir.

Bu örnekteki üç senaryo şunlardır (ekrandan, 00:31:35):

1. **Metin görüntüleme:** Sistemin amacı metni göstermektir. Dosya okunur, satırlar okunur, pencerede gösterilir ve kulbun boyutu dosya boyutuna göre ayarlanır.
2. **Kaydırma çubuğunu hareket ettirme:** Çubuk hareket ettikçe dosyanın farklı yerleri okunmalı ve Göstericide gösterilmelidir. Burada da sorumluluklar vardır, bu yüzden ayrı bir senaryodur.
3. **Göstericiyi yeniden boyutlandırma:** Pencere 1 ile 100 satır arasında yeniden boyutlandırılabilir. Bu, dosyanın bir kısmının okunmasını ya da görünürden kaldırılmasını gerektirir.

Sistemin davranışları, kullanıcıya vereceği tepkiler, beklenen olaylar ve bu olaylara verilecek tepkiler **use case modellemesiyle** ifade edilir. Ana yapısal elemanlar ve sistemin davranışları yazılıp örneğin bir sınıf diyagramıyla çizildiğinde **analiz modeli** elde edilir. Hoca, analiz modelinin problemi paydaşlarla birlikte çalışacak ve anlaşacak şekilde göstermeyi de kolaylaştırdığını söyler.

Hoca başlangıç işlemlerine de değinir. Bu örnekte uygulamanın düzgün başladığı varsayılır. Büyük sistemlerde başlangıç ayrı bir dert olabilir: kim kime nasıl hizmet verecek, hangi sırayla oluşturulacak? Örneğin bir API Gateway yapılırken mesajların servislere nasıl dağıtılacağı, kuyruk mekanizmasının ne olacağı, önce kuyruğun mu servisin mi çalıştırılacağı gibi kararlar verilir. Bunlar da ayrı bir senaryo gibi modellenebilir.

6. Sınıf diyagramı: özellikler, işlemler, parametreler

6.1 Sınıf kutusu

UML sınıf diyagramında bir sınıf üç bölmeli bir dikdörtgenle gösterilir: bileşenin adı, özellikleri ve davranışları (işlemleri). Hoca bunun en basit hâl olduğunu, beş bölmeli sınıf kutularının da görülebileceğini ve ilerleyen haftalarda örneğini göstereceğini söyler.

Bu aşamadaki üç sınıf **kavramsal sınıflardır**, yani uygulamanın aday sınıflarıdır. Bunların gerçek yazılım modelinde nerede ve nasıl duracağına, nasıl gerçekleştirileceğine girilmez. Yalnız “bu problemi çözmek için bu bileşenlere ihtiyacım var, yapısal elemanlarım bunlar” denir.

6.2 Özellikler

Özellikler, görülebildiği ve ifade edilebildiği kadarıyla yazılır. Örneğin göstericinin satır sayısı, kaydırma çubuğunun kulp boyutu ve konumu, dosya yöneticisinde dosyanın adı, boyutu ve satır sayısı.

6.3 İşlemler

İşlemler, sistemin kullanıcıyla etkileşimini sağlayan davranışlardır. Analiz aşamasında metot gerçekleştirmesiyle ilgilenilmez: nasıl kodlanacağı, kimin kime hizmet vereceği, hangi kalıplarla yapılacağı bu aşamanın işi değildir. Yalnız sistemin kullanıcıya vereceği tepkiler ortaya konur.

- **Metni göstermek** bir tepki değildir. Ekranda yalnız metin gösterilir.

- **Kaydırma çubuğunu hareket ettirmek** bir tepki gerektirir: başka satırların gösterilmesi.
- **Pencereyi boyutlandırmak** bir tepki gerektirir: 20 yerine 50 satır gösterecek şekilde boyutlandırınca gösterilen alan değişmelidir.

Bu tepkiler sınıf kutusunun üçüncü bölmesine yazılır. Gösterici sınıfına `boyutlandır(yeniBoyut)`, KaydırmaÇubuğu sınıfına `kaydır(yeniKonum)` işlemleri konur; DosyaYöneticisinin bu aşamada işlemi yoktur (ekrandan, 00:39:50).

6.4 Parametreler ve dönüş değerleri

Slayttaki "İşlem Parametreleri" başlığı şu soruları sorar (ekrandan, 00:39:50):

- Göstericinin boyutları neler olmalı?
- Tablada kulbun pozisyonu nedir?
- Boyutlandır davranışının geri dönüş değeri ne olabilir? İşlem dönüş değeri için değil, etkisi için yapılıyorsa "void". Sadece UML veri tipleri kullanılır, gerçekleştirme ayrıntılarına girilmez.
- Kaydır davranışının geri dönüş değeri ne olabilir?

Boyutlandırmada bir parametre gerekir: yeni boyut. Satır sayısı özelliği 20 iken 50 ya da 80 yapılmak istenir. Kaydırmada da kulbun taşındığı yer bilinmelidir. Örneğin kulp sona çekildiyse dosya yöneticisinden son satırları istemek gerekir.

Hoca veri tipleri için şunu söyler: UML'in standart veri tiplerini (string, integer, void) kullanın. Özel veri tiplerine ve "falanca sınıf tipinde dönüş değeri" gibi gerçekleştirme ayrıntılarına girmeyin.

6.5 Tartışma: `kaydır` bir değer döndürmeli mi?

Hoca kaydırma davranışının dönüş değerini sınıfa sorar.

Soru (hoca): Kaydırma davranışında geri dönüş ne olabilir?

Cevap (bir öğrenci): Kaydırma sonucunda konum değiştiği için verinin tekrar alınması gerekir. Konuma göre hangi satırların gösterileceği (atlanacak ve alınacak satırlar) bulunup satırların kendisi görüntüleyiciye döndürülmelidir.

Soru (hoca): Bunu kaydırma çubuğu mu yapmalı?

Cevap (öğrenci): Kaydırma çubuğu konumunu döndürmeli, arada hesaplama yapan bir metoda ya da katmana ihtiyaç var.

Soru (hoca): Kaydırma çubuğu geri hiçbir şey döndürmese ne olur?

Cevap (öğrenci): O zaman konum değişikliği olayı nasıl alınacak? Dosya yöneticisi de görüntüleyici de bunu bilmeyecek. Bir şekilde kaydırma çubuğunun olayına erişmek gerekir.

Hoca buradan şu sonuca gider: kaydırma çubuğu "bana bir şey oldu, konumum artık şurası" diye bir duyuru (yayın) yapmalıdır. Birilerinin de bunu dinlemesi gerekir. Örneğin buradaki denetleyici rolündeki pencere "kaydırma çubuğuna bir şey oldu, yeni satırlar göstermem gerekiyor" diye fark edip tepki verir. Duyuru, `kaydır` işleminin dönüş değeriyle değil, kaydırma çubuğunun kendi içindeki dinleyiciler listesine haber vermesiyle yapılabilir. Öğrenci de bir olay tetiklenmesi gerektiğini, konuma diğer bileşenlerden zaten erişilebileceğini söyler; hoca bu durumda dönüş değerinin gerekmediğini belirtir.

Hoca iki kararın da doğru olabileceğini, ama birinin daha iyi çalışabileceğini, örneğin daha esnek olabileceğini söyler:

- **Dönüş değeriyle çözüm:** Bileşenler sürekli kaydırma çubuğuna gidip bir şeyler sormak zorunda kalır. Bağımlılık artar.
- **Olay yayınıyla çözüm:** Yarın yeni bir bileşen eklenirse o da yayını dinlemeye başlar. Bazı bileşenler "bu beni ilgilendirmez" der, bazıları kendi metotlarını tetikler.

  Hocanın vurgusu: Tasarımda kararlarınızı hep "gelecekte bu bana neye mal olur, bir şey değiştirmem gerekirse bunu nasıl yapmama yol açar" diye düşünerek verin.

> **Ek bilgi (derste anlatılmadı):** Hocanın tarif ettiği “duyuru yapan bileşen ve onu dinleyen bileşenler” yapısı, nesneye yönelik tasarımda gözlemci (Observer) kalıbı olarak bilinir. Yayın yapan nesne (subject) dinleyicilerinin bir listesini tutar ve durumu değişince hepsine haber verir. Yayın yapan nesne dinleyicilerin somut türünü bilmez; bu yüzden aralarındaki bağımlılık azdır.

6.6 1–100 kısıtı UML’de gösterilemez

Gösterici satır sayısı 1 ile 100 arasında olmalıdır, yani 101 değerini alamamalıdır. Bu, bileşenin bir özelliği (attribute) üzerindeki bir kısıttır. Hoca bunun UML diyagram notasyonuyla ifade edilemeyeceğini söyler.

Ayrıca gerçek hayatta kullanılan kütüphane büyük olasılıkla “satır sayısı” gibi soyut bir değer döndürmez; piksel konumu ya da göreli ekran çözünürlüğüne göre bir değer döndürür. Bu değeri 20, 21, 54, 100 gibi bir tamsayıya çevirmek ve 100’ü geçmemesini sağlamak tasarımcının işidir. Hoca, böyle ayrıntıların mutlaka çıkacağını, ama analiz modelinde bunlara fazla girilmemesi gerektiğini söyler. Analizde odak sistemin davranışları, tepkileri ve bileşenlerin kim kime hizmet vereceğidir. 1–100 gibi kısıtlar UML’de OCL kullanılarak yazılabilir.

7. Görünen özellikler (percept) ve paydaşlar

7.1 Percept

Slayttaki tanım (ekrandan, 00:46:30): uygulamaya ait özelliklerden kullanıcının görebildiklerine **algı (percept)** adı verilir. Algılanan her kavram bulunup özellik olarak modellenmeye çalışılır. Göstericinin görünen özellikleri:

- Kulbun pozisyonu.
- Göstericinin boyutları.
- Kulbun boyutu. Değişik boyutlu bir dosya gösterilirse ya da göstericinin boyutları değiştirilirse bu da mutlaka değişmelidir.

Hoca bunlara unutulan bir özelliği ekler: metnin kendisi. Pencerenin içinde bir metin vardır ve bu metin (string) de pencerenin bir özelliğidir.

Hoca bu noktada bir tasarım tavsiyesi verir. Bunların yüzde yüz doğru olacağı diye bir kural yoktur, ama mümkün olduğunca herkes kendi bildiğini tutmalı, uzmanı olduğu işlere bakmalı, fazlasını bilmemelidir.

> **Hocanın vurgusu:** Ne kadar çok şey bilirseniz o kadar çok şey yapmak zorunda kalırsınız; yazılım bileşenleri dünyasında da bu böyledir. Tasarımı her bileşenin belli işleri yapacağı biçimde yönlendirin. Bu bir kural değil, tavsiyedir.

7.2 DosyaYöneticisinin görünen özelliği

Dosya yöneticisi kullanıcıyla doğrudan etkileşen bir bileşen değildir, ama metin içeriğini sağlar ve dosya sistemine erişir. Onun görünen özelliği belgenin kendisidir: işletim sistemi aktörü tarafından sağlanan, okunan dosyanın içeriği.

7.3 İç ve dış paydaşlar

Hoca bir soru daha önerir: sisteme kuş bakışı, dışarıdan ve yukarıdan bakıldığında hangileri iç paydaş, hangileri dış paydaştır? Bu soru sistemin aktörlerini bulmayı kolaylaştırır.

- **Kullanıcılar** sistemden fayda sağlar: metni görür ve okurlar. Dış paydaştır.
- **Dosya sistemi ya da işletim sistemi** de dış paydaştır, çünkü belgeyi sisteme o verir; sistemin içinde değildir.

Buraya kadar bulunanlar, problem üzerinde konuşa konuşa ve gözlemleyerek ortaya konan aday sınıflar, bunların kullanıcıya sunacağı hizmetler ve bu hizmetler için gereken görünen özelliklerdir. Hoca bunun işin kolay kısmı olduğunu, zor kısmın aralarındaki ilişkileri bulmak olduğunu söyler.

8. Bileşenler arası ilişkiler

8.1 Üç ilişki türü

Slayttaki ifade (ekrandan, 01:03:00): buraya kadar sınıflar, davranışları ve görünen özellikleri oluşturuldu; bu kolay kısımdı. Zor olan bileşenler arası ilişkileri ortaya çıkarmaktır. UML analiz modelinde 3 ilişki türü bulunur:

- İşbirliği (association)
- Birleşme (aggregation)
- Genelleme (generalization)

İlişkileri bulmak için sorulacak soru şudur: hangi bileşen hangi kullanıcı eylemini karşılamaktan sorumludur? Bu soruya, yazılan kullanım senaryoları ve bu senaryolardaki sorumluluklar cevap verir.

8.2 Senaryolardaki alt eylemler

Boyutlandırma ve kaydırma davranışlarının alt eylemleri vardır:

- **Boyutlandırma:** Farklı sayıda metin satırına ihtiyaç doğar. Kulbun hem boyutu hem konumu değişir. Göstericinin kendi boyutları (perceptleri) değişir.
- **Kaydırma:** Farklı sayıda metin satırına ihtiyaç doğar. Kulbun konumu değişir. Gösterilen metin, yani pencerenin bir özelliği de değişir.

Bu alt eylemler ilişkileri oluşturmaya başlar: pencere ile kaydırma çubuğu arasında, pencere ile dosya yöneticisi arasında ilişki vardır. Hoca, önceki tartışmayı hatırlatarak tasarıma bağlı olarak kaydırma çubuğunun doğrudan dosya yöneticisiyle de konuşabileceğini söyler. Kim kimle konuşuyorsa, kim kimden hizmet alıp veriyorsa aralarında bir ilişki vardır. Bu ilişkiler, senaryolarda bileşenlerin birlikte çalışma durumlarından çıkarılır.

8.3 Görünen satır sayısı ilişkisi

Slayttaki soru (ekrandan, 01:08:15): herhangi bir anda Göstericide görünen metin satırı sayısını, pencere boyutu ve dosyadaki satır sayısının bir fonksiyonu olarak nasıl ifade edebiliriz?

- **Pencere Boyutu:** Pencerede görünen satır sayısı.
- **Dosya Satır Sayısı:** Dosyadaki satır sayısı.
- Eğer dosyada yeteri kadar satır varsa, Pencere Boyutu kadar satır gösterilir.
- Dosyanın satır sayısı Göstericiden azsa, Dosya Satır Sayısı kadar gösterilir.

Cevap:

$$\text{görünen satır sayısı} = \min\big(\text{Pencere Boyutu}, \text{Dosya Satır Sayısı}\big)$$

Hocanın örneği: pencere 80 satır gösterebiliyor, ama dosyada yalnız 5 satır var. Bu durumda $\min(80, 5) = 5$ satır gösterilir.

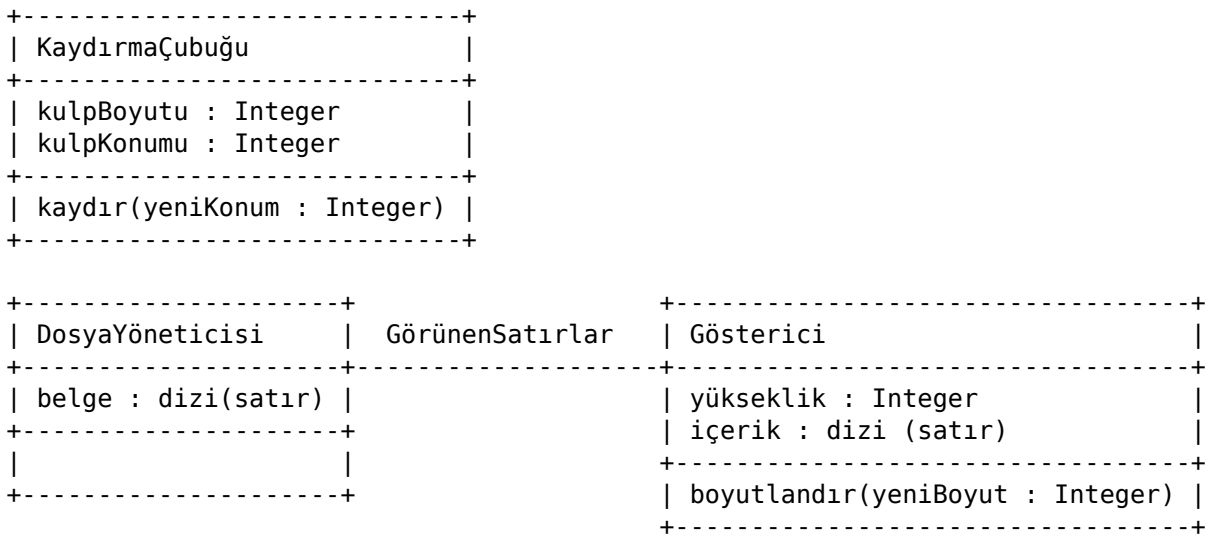
Hoca, bu değerin her zaman bu kadar basit bir minimumla bulunamayabileceğini, duruma göre farklı davranışlar da gerekebileceğini söyler. Ayrıca “bu özellik şu ikisinin minimumudur” ifadesi diyagramda gösterilemez. Bunun için bir kısıt diline ihtiyaç vardır: UML’de kullanılan **Object Constraint Language (OCL)**.

Hoca UML denince çoğu kişinin aklına sınıf ve sıralama (sequence) diyagramlarının, bazen durum (state chart) ve etkinlik (activity) diyagramlarının geldiğini söyler. Ancak bir fonksiyon sonucunda oluşan özellikler ya da “bu değer yalnız 1 ile 100 arasında olacaktır” gibi kısıtlar diyagramlarda gösterilemez. Bunlar için UML’e yardımcı olan OCL kullanılır. Ders kapsamında OCL’ye giriş yapılacak ve ileri seviyeli tasarım kısıtlarının nasıl gösterileceği görülecektir.

> **Ek bilgi (derste anlatılmadı):** OCL’de bir sınıfın her zaman sağlaması gereken kısıt değişmez (invariant) olarak yazılır. Örneğin derste geçen 1–100 kısıtı `context Gösterici inv: yükseklik >= 1 and yükseklik <= 100` biçiminde ifade edilebilir. Türetilmiş bir özelliğin değeri ise `derive:` ifadesiyle tanımlanabilir; görünen satır sayısının minimum olarak tanımlanması bu türdendir.

8.4 Şu ana kadarki model

Slayttaki "Görünen Satır Sayısı İlişkisi" diyagramı şöyledir (ekrandan, 01:10:00):



Slaytta KaydırmaÇubuğu ayrı durur; bu aşamada onunla diğer iki sınıf arasında bir ilişki çizgisi yoktur.

Hoca modeli şöyle özetler. Kaydırma çubuğunun görünen özellikleri (kulp boyutu ve konumu) ve bir davranışı vardır; davranışın içi henüz yoktur, ama bir tepki olması gerektiği bilinir. Dosya yöneticisinin belge özelliği satırlardan oluşan bir dizi olarak modellenir. Göstericinin yüksekliği (1 ile 100 arasında değişen) ve içeriği vardır, bir de boyutlandırma davranışı vardır. Görünen satırlar, belge ve içerik özelliklerini birbirine köprüleyen, dosya yöneticisi ile gösterici sınıflarını ilişkide tutan özelliktir. Yani ilişkinin kendisinin "görünen satırlar" diye bir özelliği vardır.

8.5 Hangi satırlar görünecek?

Görünen satır sayısı bilinir (örneğin 20). Hangi satırların görüneceğini ise kaydırma çubuğunun konumu belirler. Kulbun, yani çekiştirilen alanın tepesi gösterilecek ilk satırı işaret eder. Kulp tam ortadaysa ortadaki satırdan sonrası, sondaysa sondan önceki satırlar gösterilir. Gösterilecek ilk satır bulunduktan sonra üstüne görünen satır sayısı eklenerek aralık belirlenir.

Slayttaki ifade (ekrandan, 01:12:20):

```
Görünecek En üst Satır + Görünen Satır Sayısı

Bunu belirlerken dayanak aldığımız özellikler:
  Gösterici:       Boyut
  DosyaYönetici:   belgeBoyutu
  KaydırmaÇubuğu:  kulpKonumu
```

Boyut Göstericiden, dosyanın boyutu DosyaYöneticisinden, konum KaydırmaÇubuğundan alınır. Böylece üç bileşen arasında üçlü (ternary) bir ilişki oluşur: **görünür ilişkisi**. Gösterici, DosyaYöneticisinin sağladığı ve KaydırmaÇubuğunun konumu tarafından belirlenen satırları gösterir.

> **Hocanın vurgusu:** Üçlü ilişkiler genellikle sevilmez ve mümkün olduğunca kaçınılır. Ne kadar çok bağlı olursanız o kadar değişemez hâle gelirsiniz; değişikliklere uyum sağlayamaz ve gitgide yönetilemez olursunuz. Bu örnekte kaydırma çubuğu ile pencere neredeyse ayrılmaz parçalar olduğu için durum çok göze batmaz. Ama kendi bileşenlerinizi modellerken üçlü ilişkileri ikililere çevirmeye çalışın.

8.6 Kulp boyutu ilişkisi

Slayttaki ifade (ekrandan, 01:14:30): bileşenler arası ilişkiler şeklinde kulp boyutu nasıl ifade edilir? `kulpOranı = 1` ise kulp tüm tablayı kaplar. Genel ifade:

$$\text{kulpOranı} = \frac{\text{GörünenSatırSayısı}}{\text{dosyaBoyutu}}$$

Hocanın sayısal örneği:

- Verilen: dosyada 1000 satır var, pencerede 100 satır görünüyor.
- İşlem: $100/1000 = 0{,}1$.
- Sonuç: kulp tablanın %10'unu kaplar. Tabla toplam 100 birimse kulp 10 birimdir (tablanın onda biri).

Dosya boyutu DosyaYöneticisinden, görünen satır sayısı pencereden gelir. Bu ikisinin oranından kaydırma çubuğunun görsel boyutu bulunur. Yani bu boyutu hesaplamak için DosyaYöneticisi, Gösterici ve KaydırmaÇubuğu arasında yine üçlü bir ilişki kurulur. Hoca buna **kulp boyut oranı ilişkisi** adını verir ve bu ilişkinin de diyagramda gösterilmesi gerektiğini söyler.

9. Uç durumlar ve analiz modelinin özeti

9.1 Belgenin satır sayısı sıfırsa?

Hoca bu kadar basit bir örnekte bile gözlerin açık ve tetikte olunması gerektiğini söyler ve sorar: belgenin satır sayısı sıfırsa ne olur? Dosya boştur; yalnız adı vardır, içerik yoktur.

Kulp boyutunu ve görünecek satırları hesaplarken dosya boyutu kullanılmıştı. Kulp oranı formülünde dosya boyutuna bölündüğü için boş dosyada sıfıra bölünür ve program bir istisna (exception) ile çöker. Hoca, böyle bir istek geldiğinde her şeyi tasarlasanız bile çoğu kişinin sıfır boyut durumunu unutacağını ya da göz ardı edeceğini söyler. Bu özel durumlar bazen sonradan ortaya çıkar.

> **Hocanın vurgusu:** Uç nokta problemlerinde azami dikkat gösterin. İstisnalar elbette yakalanabilir, ama sıfır boyutlu bir dosyayı okumak gibi çok basit bir ayrıntı bütün uygulamanın çökmesine yol açabilir.

Hoca ayrıca şunu hatırlatır: 100 satır ve 1000 satır gibi değerler gerçekte piksel değerleri olacaktır ve tamsayıya dönüştürülmeleri gerekir. Dosyanın sonundayken pencerenin büyütülmesi durumu da (bkz. 4.3) bu tür kararlardandır.

9.2 Analiz modelinde sorulacak temel soru

Bileşenleri, özelliklerini ve davranışlarını adım adım koyup davranışlardan çıkan ilişkileri bağladıkça, ilk bakışta görülemeyen ayrıntılar (boş belge, dosyanın sonunda yeniden boyutlandırma) görünür hâle gelir. Bunlar çözülmeli ve belgelenmelidir.

Hoca yapılan işi şöyle özetler: basit bir metin görüntüleyici için analiz modeli oluşturuldu, problem henüz çözülmedi; problem tasarımda çözülecektir. Analizde sorulacak temel soru şudur: herhangi bir metin görüntüleyici uygulamasının çözmesi gereken anahtar tasarım problemleri nelerdir, ne gibi davranışları, tepkileri ve hizmetleri olmalıdır?

Adımlar sırasıyla şöyledir:

1. Bir hizmeti sunmak için hangi bileşenlere, hangi yapısal parçalara ihtiyaç olduğunu ortaya koymak.
2. Bunun için hangi davranışların gerektiğini, yani senaryoları, hizmetleri ve sorumlulukları belirlemek.
3. Bu sorumlulukları yerine getirmek için bileşenlerin hangi özelliklere ve davranışlara (metotlara) sahip olması gerektiğini bulmak ve sorumlulukları bileşenlere atamak.
4. Bir senaryo içinde kimin kime hangi hizmeti vereceğini, yani ilişkileri belirlemek.
5. Bunları UML ve OCL ile biçimsel olarak belgelemek.

Bundan sonraki adım, modeli belirli bir mimariye göre tasarlamaktır. Problemin doğasına, çalışacağı ortama ve hizmet vereceği duruma göre bir mimari stil ya da yaklaşım seçilir.

10. Genel tasarım kavramları: tasarım, mühendislik, zanaat, sanat

10.1 Tasarım her yerdedir

Slayttaki örnekler (ekrandan, 01:20:00): Uluslararası Uzay İstasyonu ve haftaya planlanan aile yemeği. Bir aile yemeğinde bile neler yapılacağı, ne içileceği, kimin nereden geleceği, sofrada kimin nerede oturacağı tasarlanır. Uluslararası Uzay İstasyonunda birçok ülke birlikte çalışır. Biri metrik, biri imparatorluk (inç, mil) ölçü sistemini kullanır. Birlikte çalışacak bir aletin parçalarının "tık" diye oturması gerekir ve ölçü dönüşümü bile başlı başına bir sorundur. Slayttaki sonuç: bu ders yazılım mimarisi ve tasarımına odaklansa da genel tasarım kavramlarının çoğu yazılımda da rol oynar.

10.2 Tasarım: bilinçli ve amaçlı planlama

Tasarımın kelime anlamı **bilinçli ve amaçlı planlama yapmaktır**. Ne yaptığının farkında olarak belirli bir amaca hizmet edecek şekilde planlama yapan tasarım yapmış olur. Amaç muğlaksa ("güzel bir şey olsun") ya da iş bilinçli yapılmıyorsa düzgün bir tasarım olmaz.

Hocanın örneği: "Mikroservis diye bir şey çıkmış, hadi orta katmanı mikroservise taşıyalım" ya da "konteyner çıkmış, biz niye konteynerde çalışmıyoruz" demek, belirli bir bilinç yoksa ve iş moda olduğu için yapılıyorsa düzgün tasarım değildir. Tasarım yaklaşımları belirli problemleri çözmek için vardır; hepsini çözen bir yaklaşım yoktur. Mikroservisin ya da konteynerin uygun olmadığı alanlar olacaktır. Eski ya da hantal bulunan monolitik orta katmanın daha doğru çözüm olduğu problemler de olabilir.

> **Hocanın vurgusu:** Bilinç ve amaç, tasarımlarda mutlaka bulunması gereken iki esas unsurdur.

10.3 Mühendislik, zanaat ve sanat

- **Mühendislik:** Bir üretimdir; ortaya ürün çıkarmaktır. Becerili ya da sanatsal bir üretimdir. Becerili, ürünün işini düzgün yapması, kendisinden bekleneni ortaya koymasıdır; sanatsal, göze hoş gelmesi ya da ergonomik olmasıdır. Mühendislik çalışmalarında bilimsel ve matematiksel temeller mutlaka uygulanır.
- **Zanaat:** Bir meslektir, ama yetenek isteyen bir meslektir. Yetenek de deneyimden gelir. Hocanın örneği bir marangozdur. Herkes testereyi alıp tahtayı kesebilir, ama zanaatkâr kese kese ve deneyimledikçe bunu bir yeteneğe dönüştürür. Yaptığı sandalye "evladiyelik" olur: göze hoş görünür, rahattır, sağlam ve dayanıklıdır; ne sallanır, ne gıcırdar, ne kırılır. Tasarımı ya da mühendisliği düzgün yapılmamış bir sandalye ise gıcırdar, zamanla ek yerlerinden ayrılıp çatlar. Hoca, yazılım sektöründe bilgisayar mühendisi olmadan da kendini yetiştirenlerin bu işi yapabildiğini, işin içine burada zanaatin girdiğini söyler. En iyisi mühendisliği ve zanaati birleştirmektir.
- **Sanat:** Beceri, tat ve hayal gücünü estetik nesnelerin üretiminde kullanmaktır.

Hoca bu tanımların üretimde göz önünde tutulması gerektiğini söyler: tasarımlar bilinçli ve amaçlı olmalı, mühendislik işin içinde olmalı, mümkünse deneyim (zanaat) konuşmalı ve ürün mümkün olduğunca estetik olmalıdır. Örnek olarak akıllı telefonları verir: bazı arayüzler ve uygulamalar insanlara daha sevimli, çekici ve kolay gelir; kullanımı zor ya da görünümü kötü olanlar zamanla piyasadan silinir.

11. Yazılım tasarımı ve programlama

Slayttaki "Yazılım Tasarımı ve Programlama arasındaki farklar" (ekrandan, 01:30:00):

- **Ölçek:** Büyük bir çözüm gerektiren büyük bir problemle uğraşılır. Programlarken 100, 1000 ya da 10000 satır kodla problem çözülür. Uluslararası Uzay İstasyonunda 30 milyon satır kod vardır.
- **İşlevsel olmayan gereksinimlere önem vermek:** Programlarken en fazla performansla ilgilenilebilir. Tasarımda işlevsel olmayan gereksinimler arası denge sağlanmalıdır.

Hoca, 30 milyon satır kodun birlikte düzgün çalışıp hizmet verebilmesi için çok düzgün bir tasarım gerektiğini söyler. Tasarımda hem işlevsel sorumluluklar ortaya konmalı hem de işlevsel olmayan gereksinimler karşılanmalı, ikisi arasında en iyi denge kurulmalıdır.

Yazılım tasarımının tanımı: Problemin işlevsel gereksinimlerini yerine getirirken işlevsel olmayan gereksinimleri de atlamadan yazılım üretimi gerçekleştirildiğinde bir yazılım tasarlanmış olur.

Burada her zaman bir şeyden ödün vermek gerekir: hangisinden, neden ve nasıl (örneğin performans mı, kaynak kullanımı mı)? Bu kararları tasarımcı verir ve çözümün tasarımı ona aittir.

11.1 Örnek: performans için tasarım (hava tahmini)

Hoca performansın öncelik olduğu bir örnek verir. Bir hava tahmini uygulaması coğrafi bir ızgara (grid) üzerinde çalışır ve hücrelerin komşulukları bilinir. Bir (x, y) koordinatında yağmur yağdıran bir bulut saatler içinde sağa doğru (x ekseninde $x + n$ biçiminde) ilerler. Hocanın günlük hayattan örneği Balkanlardan gelen yağıştır: bulut önce Edirne'nin, bir saat sonra Kırklareli'nin üzerindedir. Buradan bir saat sonra İstanbul'un Avrupa yakasına, iki saat sonra Anadolu yakasına geleceği tahmin edilebilir. Uygulamanın hızlı, gerçek zamanlı çalışması beklenir.

- Böyle bir veri yapısı için dizi mi, nesne koleksiyonu mu kullanılmalı? Hocaya göre dizi performansı artırır, çünkü aynı türdeki veriler üzerinde işlem yapmayı kolaylaştırır. Nesneler ve nesne koleksiyonları hem daha fazla bellek hem de performans kaybı demektir.
- Yeni bir model çıkar ve komşuluk dikdörtgen değil, bal peteği gibi altıgen (hexagonal) olur: her hücrenin 4 değil 6 komşusu vardır. Bu durumda dizi kullanılamaz, başka bir yapı gerekir ve performanstan kayıp yaşanır.

Ders şudur: tasarım yaparken "komşuluk her zaman dikdörtgen ya da matris şeklinde mi olacak, yoksa değişebilir mi?" diye sorgulamak gerekir. Yarın değişebilecek bir durum varsa veri yapıları ve nesneler buna göre tasarlanır. Yazılım dünyasında her şey aynı anda en iyi olamaz; bir şeyden mutlaka ödün verilir.

12. Mimari tasarım ve detaylı tasarım

Mimari tasarım, yazılımın modül ve bileşenlerine, davranışsal yönleriyle belirlenen sorumlulukları atamaktır. Yani programın parçalarını bileşenler hâlinde birleştirmek, bu bileşenlere davranış biçiminde sorumluluk yüklemek ve bileşenlerin birlikte nasıl çalışıp etkileşeceklerini belirlemektir. Bileşenlerin bir araya gelme ve hizmet verme biçimleri düşünüldüğünde mimari tasarım modellenmiş olur.

Detaylı tasarım ise davranışların tek tek, veri yapıları ve algoritmalarıyla birlikte tanımlanmasıdır: metotlar nasıl çalışacak, içlerinde ne olacak? Ders detaylı tasarıma fazla girmez. Detaylı tasarım şu araçlarla gösterilebilir:

- Sözde kod (pseudo code),
- Yapısal programlama ifadeleri,
- Akış diyagramları,
- Karar tabloları.

Mimari tasarımda ise bu derste daha çok UML kullanılacaktır.

13. Yazılım tasarımı yaklaşımları ve ortak üç yön

Tasarım yaklaşımları "bir sistemi en iyi nasıl yapılandırabiliriz?" sorusunu cevaplar. Hocanın saydığı yaklaşımlar:

- Nesneye yönelik tasarım yaklaşımı.
- Yapısal tasarım yaklaşımı.
- Rol tabanlı tasarım yaklaşımı.
- Belirli bir sınıftaki yazılımlara odaklanan yaklaşımlar. En güzel örneği gerçek zamanlı (real-time) sistemlerdir; bu sistemlerde birçok şey çok farklı biçimde modellenir.

- Uygulamanın yalnız bir kısmına odaklanan yaklaşımlar. Örneği UI/UX tasarımıdır: yalnız kullanıcıyla etkileşen arayüzlerle ve kullanıcının yaşayacağı deneyimle ilgilenir. Hoca, mobil dünyada bunun belki de en önemli nokta olduğunu söyler. Android ve iOS tasarım yaklaşımları çok farklı noktalardan başlamış, zamanla birbirine yaklaşmıştır (butonların yerleşimi, basma biçimleri); hâlâ farklar vardır ama ölümcül değildir.

Yaklaşım ne olursa olsun (C ile yapısal programlama, Java ile nesneye yönelik geliştirme, Spring, MVC) tüm yaklaşımlarda ortak olan 3 yön vardır (ekrandan, 01:40:00):

- **Tasarım yöntemi:** Tasarımcıların bir problemi çözmek için uyguladıkları sistematik ve sıralı işlem adımlarıdır. Genellikle problemi göstermek için belirli bir yol önerir. Örneğin nesneye yönelik tasarımda problem işbirliği yapan nesneler şeklinde tanımlanır. Yöntem, tüm takıma düşünce ve aktivitelerini belirli şekilde yapmaları için disiplin sağlar.
- **Tasarım gösterimi:** Tasarımı görsel ya da yazılı ifadelerle ifade etmektir. Nesneye yönelik tasarımda UML ve OCL, yapısal programlamada akış diyagramları buna örnektir.
- **Tasarım geçerleme:** Doğrulama (validasyon), gözden geçirme (review). "Yaptım, bitti" denmemeli, yapılan iş gözden geçirilmelidir. Hoca örnek olarak metin görüntüleyicideki sıfır boyutlu dosya durumunu hatırlatır.

  Ders sonrası not: Altyazıda (01:38:52) hoca "Spring diyecektim, Swing dedim" diyerek kendini düzeltir. Hocanın kastettiği, Java ile Spring çatısı kullanılarak yapılan geliştirmedir.

14. Tasarım ikilemleri ve geçerleme

14.1 Yazılım tasarım ikilemleri

Slayttaki ikilemler (ekrandan, 01:41:00):

- Mimar ya da tasarımcı olarak kararlar vermek gerekir: işler yukarıdan aşağıya mı (bütünden özele), aşağıdan yukarıya mı (parçaları birleştirerek bütüne), içten dışa mı yapılacak?
- Yordamlar ve fonksiyonlar mı düşünülecek, kavramlar ve davranışlar mı?
- Yazılım üretiminde kaçınılmaz sorun: **tasarım zaman alır**. Düşünmek vakit harcatır. Uzun dönem yönetilebilir bir yapı mı kurulacak, kısa süreli takvimli bir iş mi yetiştirilecek? İyi bir tasarım mı yapılacak, hızlıca bir ürün mü çıkarılacak?
- Hangi araçlar kullanılacak? Ekibin aşina oldukları mı, teknolojinin sunduğu ya da dayattığı araçlar mı?

Hoca, yazılım tasarlamanın basitçe hep ikilemleri çözmek olduğunu söyler.

14.2 Geçerleme

Geçerleme (gözden geçirme), bütün tasarım yaklaşımlarında bulunması gereken üçüncü adımdır. Bir yöntem tanımlandıktan sonra tasarımın gereksinimlere uygun olup olmadığı gözden geçirilmelidir. Bunu genellikle bir tasarım gözden geçirme ekibi (design review team) yapar. Böyle bir kaynak yoksa bu rolü tasarımcının üstlenmesi gerekir. CASE araçları ve IDE'ler de birçok kontrolü yapabilir.

Soru (hocanın öngördüğü itiraz): Zaten program yazılacak, otomatikleşen regresyon testleri gibi birçok test yapılacak. Tasarımı hemen gözden geçirmeye gerek var mı, iş bitince gözden geçirilse olmaz mı?

Cevap (hoca): Mümkünse tasarımı yaptıktan sonra gözden geçirin. Bir problem ne kadar erken tespit edilirse çözüm maliyeti o kadar düşer; o kadar ucuz ve hızlı çözülür. Kritik hatalar çok geç fark edilir ve tasarımın ya da uygulamanın büyük kısmını etkileyen değişiklikler gerekirse maliyet ve takvim çok daha zorlayıcı olur.

Önemli hususlar:

- **Geçerleyenler bağımsız olmalıdır.** Gözden geçirmeyi yapanlar tasarlayanlardan farklı olmalıdır. Başkasının çözümüne bakan kişi "burada niye böyle yaptın, şunu atladın mı, şöyle olsa ne olur?" diye sorabilir. Tasarlarken düşünülmeyen ayrıntılar aynı kişi tarafından gözden geçirirken de doğal olarak düşünülemez.

- **Tasarım yöntemine bağımlılık:** Örneğin yapısal tasarımda sonuçları metriklere eşleyen bir kurallar kümesi vardır. Metrikler yönteme göre değişir; nesneye yönelik tasarımda başka, yapısal tasarımda başka metrikler kullanılır. Ama sonuçların ölçülebilir olması beklenir.
- **Ne zaman yapılır:** Tasarlarken ya da bitince. Günlük ya da haftalık gözden geçirme ve düzeltmeler yapılabilir ya da proje takviminde kilometre taşları (milestone) oluşturulup bu noktalarda daha büyük gözden geçirmeler yapılabilir. Hoca, Scrum tarzı yöntemlerin günlük ve haftalık gözden geçirmeleri daha sistematik ve hızlı yapmak için araçlar sunduğunu söyler.

14.3 Dikkat edilecek diğer unsurlar

Hoca bunların kesin cevabı olmadığını, biraz muğlak olduğunu belirterek şu unsurları sayar:

- Mimari tasarım ile detaylı tasarımın sınırı nerededir? Biri nerede biter, diğeri nerede başlar? Mimari tasarım ne kadar ayrıntılı olmalıdır?
- İşlevsel ve işlevsel olmayan gereksinimler arasındaki dengede hangisine ne kadar ağırlık verilmelidir?
- "Ne" ile "nasıl" dengesi: tanımlama (analiz) ne kadar, tasarlama ne kadar olacak?
- Hazır kaynaklar ya da önceden hazırlanmış çözümler kullanılmak istenirse sorulacak soru: uygulamanın özelinde tasarımı etkileyen faktörler nelerdir? Bu faktörler bilinir ve öngörülürse hazır çözümler daha rahat kullanılabilir.

15. Belgelendirme

Büyük sistemler çok karmaşıktır. Sistem ne kadar büyük ve karmaşıksa belgelendirmeye ihtiyaç o kadar artar. Hoca çoğumuzun belgelendirmeyi göz ardı ettiğini, yorum yazmanın bile zor geldiğini söyler. Bütün emek yalnız geliştirmeye harcanırsa sistem bir süre iyi çalışır, ama bakım dönemine girdiğinde ya da bir şeylerin değişmesi gerektiğinde işler karışır. Belgelendirme ne kadar iyiyse bu dönüşümler o kadar kolay olur.

Tasarım belgelendirmesi için ISO ve IEEE standartları ve askerî projelerde kullanılan standartlar gibi biçimsel yöntemler vardır. Ama basit notlar, sunumlar ve yorumlar bile tasarımı belgelemek için çok değerli olabilir. Belgelendirme iletişimi kolaylaştırır. Hoca, bir yıl sonra kendi koduna dönen birinin büyük olasılıkla "ben burada ne yapmışım, bu değişkeni niye böyle tanımlamışım?" diyeceğini söyler. Bu, kod düzeyinde bile belgelendirmenin önemini gösterir.

Slayttaki "Geleneksel Tasarım Belgeleri" (ekrandan, 01:51:10):

- Sistemi bir araya getiren bileşenler: sorumlulukları, veri akışları.
- Kontrol akışı.
- Performans kriterleri.
- Kaynak kullanımı (hafıza, harici birimler, bant genişliği, …).
- Bunlar yetmiyorsa, IEEE 1016 standardı: tasarımcı kim, parçalar arası bağımlılıklar neler, gizli varsayımlar var mı, işlevsel olmayan gereksinimler arası ödünleşimlere nasıl karar verildi, kullanıcı, teknoloji ve donanım varsayımları neler, …

  Ders sonrası not: Altyazıda standardın numarası "IEEE 10.16" ve "ITAQ16" olarak geçer. Slayttaki doğru ad IEEE 1016'dır; bu fark Whisper hatasıdır.

  Hocanın vurgusu: Tasarımlarınızı mutlaka gerekçelendirin. "Şunu şöyle yaptım, bu sebeple yaptım; hız ile boyut arasında şu seçimi şuna göre yaptım" diye seçeneklerin neden seçildiği açıkça anlatılırsa hem siz hem de sizden sonra gelenler rahat eder.

16. Tasarımda anahtar kavramlar ve tasarım felsefesi

16.1 Kavramsal bütünlük

Slayttaki iki alıntı (ekrandan, 01:52:25):

- **Descartes:** Farklı ellerden çıkan çok parçalı işlerde mükemmellik, tek bir ustanın elinden çıkana oranla çok az görünür.

- **Fred Brooks (The Mythical Man-Month):** Bağımsız ve koordine olmamış kavramlar yerine bir tasarım fikir kümesinden çıkmak kaydıyla, alışılmadık özellikler ve gelişmeler sağlayan sistemler iyidir.

Hoca birinci alıntıyı şöyle yorumlar: bir ekip gibi farklı ellerden tasarım çıkıyorsa ürünün mükemmele yakın olması daha zordur; tek bir usta yaparsa sonuç daha iyi olur. Bu yüzden ekipte uzlaşı olmalı, herkes aynı dili konuşarak benzer bir tasarım çıktısı üretmelidir. İkinci alıntı ise dağınık çalışma, yani herkesin kendi kafasına göre çalışması yerine belirli bir fikir kümesinden çıkan ve farklılık yaratan sistemlerin iyi olduğunu söyler. Yenilik getirmek ve var olanlardan farklı olmak iyidir, ama bunu yaparken belirli bir tasarım anlayışına bağlı kalmak gerekir.

16.2 Az bağımlılık ve iyi uyum

Slayttaki ifadeler (ekrandan, 01:55:50):

- **Az bağımlılık:** Sistemin bileşenleri birbirine bağımlıdır. Başarılı çalışma için iki bileşenin mümkün olduğunca az bağımlı olması istenir. Çok bağımlı olurlarsa birindeki değişiklik diğerini de değiştirmeyi gerektirir.
- **İyi uyum:** Bir bileşenin belirli bir amacı ya da hizmeti olması iyidir. İyi uyuma sahip birimleri tekrar kullanma olasılığı daha yüksektir.

Hoca metin görüntüleyiciyi hatırlatır: üç bileşen birbirine her türlü bağımlıydı ve sürekli birbirleriyle konuşmak zorundaydı. Çok bağımlılık çok değişiklik demektir; bir şey değişince her şeyi ellemek gerekir. İyi uyum ise bileşenin iç uyumunun yüksek olması, tek bir iş yapıp o işi düzgün yapmasıdır. Hoca "her şeyi yapan" dev sınıflar yaratılmamasını öğütler. İyi uyumlu bileşenlerden oluşan sistemlerin bileşenleri sonraki tasarımlarda hazır olarak kullanılabilir ve toplam geliştirme süresi azalır. Hoca bu iki kavramın birbiriyle çok sıkı ilişkili olduğunu da belirtir.

Slaytta iki soru vardır: Java paketleri az bağımlılığı mı, iyi uyumu mu destekler? Java sınıf kalıtımı bağımlılığı artırır mı, azaltır mı? Hoca bunları sınıfa sorar, cevap gelmeyince araştırılmasını ve az bağımlılık, iyi uyum ve aralarındaki denge açısından düşünülmesini ister. Sorular aşağıdaki Örnek Sorular bölümündedir.

16.3 Bilgi gizleme

Slayttaki ifade (ekrandan, 01:57:10): **Bilgi Gizleme (David Parnas)**

- Belirli bir modülün yapabildiklerini soyut bir arayüz arkasında saklamak. Kullananlar sadece görünen arayüzü bilebilir.
- Gerçekleştirme ayrıntılarını değiştirebilmeyi sağlar.
- Erişim gizliliği (donanıma, veritabanına, sunucuya, …) sağlar.

Hoca, bilgi gizlemenin "bilgiyi tuttum, kimseye göstermem" anlamına gelmediğini vurgular. Kullanıcılara yalnız görünen arayüz açılır, gerçekleştirme ayrıntıları açılmaz ve gerektiğinde değiştirilebilir. Günümüzden örneği mikroservis dünyasında olmazsa olmaz kalıplardan biri olan **API Gateway**'dir. Pek çok uygulama dış dünyaya bir API sunar. Bu API soyut bir arayüzdür; arkasında hangi servislerin nasıl çalıştığı söylenmez, hizmetlere yalnız bu arayüz üzerinden erişilir.

16.4 Soyutlama

Hoca, az bağımlılığı ve iyi uyumu sağlayacak biçimde soyutlama yapmanın her zaman dikkat edilmesi gereken bir nokta olduğunu söyler. Karmaşık sistemlerdeki problemleri ayrıştırmak, belirli noktalara odaklanmak ve bir işi yaparken yalnız o adıma odaklanmak soyutlamadır. Örneğin analiz yaparken tasarım ayrıntılarıyla, tasarım yaparken de yalnız tasarımın ayrıntılarıyla uğraşılır. Farklı programlama modellerinde ve nesneye yönelik yaklaşımda soyutlama farklı seviyelerde görülür, ama her sistemde bulunmalıdır.

16.5 Estetik

Slayttaki ifadeler (ekrandan, 02:00:00):

- Aquinas (James Joyce çevirisi): Güzellik için üç şey gerekir: bütünlük, ahenk, saflık.
- Yazılım karşılıkları: bütünlük (completeness), tutarlılık (consistency), kavramsal bütünlük (conceptual integrity).
- Pascal: “Bu mektubu çok uzun yazdım çünkü daha kısa yapabilecek kadar vaktim yoktu.”
- Karmaşık gereksinimleri yerine getirecek basit ve etkin bir çözüm geliştirmek için zaman ve enerji harcamak gerekir.

Hoca bütünlüğü bütünlük, ahengi birlikte düzgün ve tutarlı çalışma, saflığı kavramsal bir bütün oluşturma olarak yorumlar. Bunlar sağlanırsa estetik bir tasarım yapılmış olur. Zaman konusunda da tasarıma ne kadar çok zaman harcanırsa ürünün o kadar iyi olmasının beklendiğini söyler.

16.6 Tasarım felsefesi

Slaytta “Tasarım Felsefesi” başlığının yanında bir ad yazar (ekrandan, 02:00:00). Yazılım tasarımı dört filozofla ilişkilendirilir:

- **Descartes, analitik geometri:** Tasarım açısından analiz ve model kullanımı demektir. Belirli modeller oluşturulursa Descartes’ın tasarım felsefesi uygulanmış olur.
- **Marx, sosyal etki:** Kullanıcı merkezli tasarım ve tasarımın sosyal etkileri; hatta kullanıcıları tasarıma dahil etmek. Tasarım kullanıcıların seveceği ve benimseyeceği biçimde yapılmalıdır.
- **Martin Heidegger, hedeflere ulaşmak için araç kullanımı:** Düzgün tasarım için CASE araçlarından ve IDE’lerin sunduğu araçlardan mutlaka yararlanılmalıdır.
- **Wittgenstein, dil oyunları:** Bir kelimenin keşfi, bir kavramla ilgili düşünmeyi sağlar. Örnekler: desktop, folder, trash, client/server, icon, firewall.

  Ek bilgi (derste anlatılmadı): Yazılım tasarımını Descartes, Marx, Heidegger ve Wittgenstein ile ilişkilendiren çerçeve büyük olasılıkla İsveçli tasarım araştırmacısı Pelle Ehn’e aittir. Ehn, katılımcı tasarım (participatory design) yaklaşımının öncülerindendir; kullanıcıları tasarıma dahil etme fikri bu yaklaşımın temelidir. Slayttaki adın da büyük olasılıkla bu olduğu düşünülebilir.

Hoca Wittgenstein maddesini açar. Bilgisayar dünyasındaki birçok kavram günlük kelimelerden gelir: “desktop” aslında ekrandır. Client/server ise biraz terstir: client istemci, server sunucudur, ama işin asıl ağırlığı genellikle sunucu (backend) tarafındadır. Firewall kelimesi “ateş geçmesin diye yapılmış duvar” demektir.

Soru (hoca): Firewall’a neden firewall denir, hiç merak ettiniz mi?

Cevap (hoca): Eski ahşap evlerin birbirine bakan yanları tuğladan örülür. Buna ateş duvarı denir; içi dolu blok tuğlalar (ateş tuğlası) kullanılır. Bir ev yandığında yangın yandaki evlere sıçramasın, daha fazla ilerlemesin diye bu duvar örülür. Firewall da güvenlik açısından bir şeyin bir yere geçememesi, diğer sistemlere sıçramaması için kullanılan kavramdır; adı buradan gelir.

> **Hocanın vurgusu:** Bileşenleri adlandırırken mümkün olduğunca anlamlı, arkasında bir neden yatan adlar bulmaya çalışın. Bir kelime belirli bir kavram ya da tasarımla ilgili düşünmenizi sağlamalıdır.

16.7 Özet: deneyim

Hocanın kapanış özeti şöyledir. Yazılım geliştirmenin en yaratıcı kısmı tasarlamak, bir mimariyi kurgulamaktır. Sistemin genel kalitesi ve hizmet kalitesi doğrudan tasarıma bağlıdır. Kaliteyi yansıtan en önemli gösterge deneyimdir (“kilometre”). Biri “yazılım mimarı oldum” deyip ilk işinde muhteşem bir şey çıkaramaz. Çok çalışarak, proje yaparak, üreterek ve hangi problemde hangi mimarinin daha iyi olacağını ortaya koyarak ilerlenir.

> **Hocanın vurgusu:** Deneyim bu işte olmazsa olmazdır. Tasarımın nasıl daha iyi yapılacağı ancak deneyimle öğrenilir.

Örnek Sorular

Soru 1 (ekrandan, 00:17:15): Swing ya da SWT gibi bir GUI kütüphaneniz olsun. MetinGörüntüleyici tasarlamak için hangi atomik GUI elemanlarına ihtiyaç duyarsınız? Checkbox, Window, Progress Bar, Combo Box, Scroll Bar, Radio Button.

Çözüm (derste verildi): Window ve Scroll Bar. Metin bir pencerede gösterilir. Dosyanın uzunluğu bilinmediği ve ekranda gösterilebilecek satır sayısı sınırlı olduğu için kaydırma çubuğu gerekir. Diğer elemanlara gerek yoktur. Metni GUI sağlamadığı için ayrıca bir DosyaYöneticisi bileşeni eklenir.

Soru 2 (ekrandan, 00:39:50): `boyutlandır(yeniBoyut)` ve `kaydır(yeniKonum)` davranışlarının geri dönüş değeri ne olabilir?

Çözüm (derste verildi): Boyutlandırma etkisi için yapılır, dönüş tipi `void`'dir; yalnız temel UML veri tipleri kullanılır. Kaydırma için iki çözüm tartışılır: yeni satırları ya da konumu döndürmek veya kaydırma çubuğunun "konumum değişti" diye bir olay yayınlaması. Hoca olay yayınını daha esnek bulur: dönüş değeriyle çözümde bileşenler kaydırma çubuğuna sürekli sormak zorunda kalır ve bağımlılık artar; yayında ise yeni bileşenler de dinleyici olarak eklenebilir.

Soru 3 (ekrandan, 01:08:15): Herhangi bir anda Göstericide görünen metin satırı sayısını, pencere boyutu ve dosyadaki satır sayısının bir fonksiyonu olarak nasıl ifade edebiliriz?

Çözüm (derste verildi): $\min(\text{Pencere Boyutu}, \text{Dosya Satır Sayısı})$. Dosyada yeterli satır varsa pencere boyutu kadar, yoksa dosya satır sayısı kadar satır gösterilir. Örneğin pencere 80 satır, dosya 5 satırsa $\min(80, 5) = 5$ satır gösterilir. Bu türetilmiş değer UML diyagramında gösterilemez, OCL ile yazılır.

Soru 4 (ekrandan, 01:14:30): Kaydırma çubuğunun kulp boyutunu bileşenler arası ilişkiler şeklinde nasıl ifade edebiliriz? 100 satır görünüyor ve dosyada 1000 satır varsa kulp tablanın ne kadarını kaplar?

Çözüm (derste verildi): $\text{kulpOranı} = \text{GörünenSatırSayısı}/\text{dosyaBoyutu} = 100/1000 = 0{,}1$. Kulp tablanın %10'unu kaplar. Oran 1 ise kulp tüm tablayı kaplar. Bu hesap DosyaYöneticisi, Gösterici ve KaydırmaÇubuğu arasında üçlü bir ilişkidir.

Soru 5 (derste sözlü, 01:15:07): Belgenin satır sayısı sıfırsa ne olur?

Çözüm (derste verildi): Kulp oranı hesabında dosya boyutuna bölündüğü için sıfıra bölme oluşur ve uygulama istisnayla çöker. Bu uç durum analizde fark edilip önlem alınmalıdır.

Soru 6 (derste sözlü, 00:28:40): Kaydırma çubuğu en alttayken (dosyanın son 20 satırı gösterilirken) pencere 40 satır gösterecek şekilde büyütülürse yazılım nasıl davranmalıdır?

Çözüm (derste verildi): Hoca iki seçenek verir: son 20 satır ve altında 20 boş satır, ya da son 40 satır. İkisi de doğru olabilir. Tasarımcı birini varsaymalı ve davranışı ona göre modellemelidir.

Soru 7 (derste sözlü, 01:32:34): Gerçek zamanlı çalışması beklenen, ızgara üzerinde bulut hareketini izleyen bir hava tahmini uygulamasında veri yapısı için dizi mi, nesne koleksiyonu mu tercih edilir? Komşuluk altıgen olursa ne değişir?

Çözüm (derste verildi): Performans için dizi tercih edilir; nesne koleksiyonları daha fazla bellek ve performans kaybı demektir. Komşuluk altıgen (her hücrenin 6 komşusu) olabilecekse dizi kullanılamaz, başka bir yapı gerekir ve performanstan ödün verilir.

Soru 8 (ekrandan, 01:55:50): Java paketleri az bağımlılığı mı, iyi uyumu mu destekler?

> **Çözüm (derste verilmedi):** Paketler, birbiriyle ilgili sınıfları tek bir birimde toplamaya yarar; bu yönüyle öncelikle iyi uyumu (cohesion) destekler. Ayrıca paket düzeyindeki erişim belirleyicileri (paket içi görünürlük) sayesinde paketin iç ayrıntıları dışarıya kapatılabilir; dışarıya yalnız `public` arayüz açılır. Bu da paketler arası bağımlılığı azaltır. Yani doğru kullanılan paketler ikisini birlikte destekler; paketleme kötü yapılırsa (ilgisiz sınıflar aynı pakette, paketler arası döngüsel bağımlılıklar) ikisi de bozulur.

Soru 9 (ekrandan, 01:55:50): Java sınıf kalıtımı bağımlılığı artırır mı, azaltır mı?

Çözüm (derste verilmedi): Kalıtım bağımlılığı artırır. Alt sınıf, üst sınıfın arayüzüne ve çoğu zaman gerçekleştirme ayrıntılarına sıkı biçimde bağlanır; üst sınıftaki bir değişiklik bütün alt sınıfları etkileyebilir. Bu yüzden kalıtım, nesneler arasındaki en güçlü bağımlılık türlerinden biri olarak görülür. Bağımlılığı azaltmak için arayüzler (interface) üzerinden çalışmak ya da kalıtım yerine bileşim (composition) kullanmak sık önerilen yoldur.

Soru 10 (derste sözlü, 02:02:34): Firewall'a neden firewall denir?

Çözüm (derste verildi): Eski ahşap evlerin birbirine bakan yanlarına, bir evde çıkan yangın yandakilere sıçramasın diye tuğla duvar örülür; buna ateş duvarı denir. Ağ güvenliğindeki firewall da bir tehdidin diğer sistemlere geçmesini önleyen engel olduğu için bu adı alır.

Dersin Toparlaması

Bu derste tasarımın bir tarifle değil, düşünerek ve yaparak öğrenildiği basit bir metin görüntüleyici üzerinden gösterildi. Gerekli bileşenler (Gösterici, KaydırmaÇubuğu, DosyaYöneticisi) "elimde ne var, neye ihtiyacım var" sorusuyla bulundu. Kısıtlar ve gereksinimler tasarımı şekillendirdi. Kullanım senaryolarıyla davranışlar, perceptlerle özellikler, senaryolardaki sorumluluklarla da ilişkiler ortaya çıkarıldı. Görünen satır sayısı (minimum), görünen satırlar (en üst satır + görünen satır sayısı) ve kulp oranı (görünen satır / dosya boyutu) gibi türetilmiş değerler ve 1–100 gibi kısıtlar UML diyagramında gösterilemediği için OCL'ye ihtiyaç duyuldu. Boş dosyada sıfıra bölme ve dosyanın sonunda pencereyi büyütme gibi uç durumlar, analizin bu ayrıntıları erken görmeye yaradığını gösterdi.

Dersin ikinci yarısında tasarımın bilinçli ve amaçlı planlama olduğu, yazılım tasarımının işlevsel ve işlevsel olmayan gereksinimler arasında denge kurmak olduğu ve her tasarımda bir şeyden ödün verildiği anlatıldı. Tüm yaklaşımlarda ortak olan yöntem, gösterim ve geçerleme; tasarım ikilemleri; bağımsız gözden geçirme; IEEE 1016 ile belgelendirme ve kararların gerekçelendirilmesi; kavramsal bütünlük, az bağımlılık, iyi uyum, bilgi gizleme, soyutlama ve estetik gibi anahtar kavramlar ele alındı. Hoca bir sonraki derste UML'e hızlıca girileceğini, nesneye dayalı dünyada UML'in neler sunduğuna, analiz tarafına ve UML ile sınıf modellerinin nasıl kurulduğuna, vakit kalırsa tasarım modeline bakılacağını söyledi.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.